Python OOP Practice Questions with Instructions

1. Create a class called `Person` with attributes `name` and `age`.
   - Write an `__init__` method to initialize both attributes.
   - Create two objects of the class and display their details using a method.
2. Create a class called `Dog` with attributes `name` and `breed`.
   - Initialize attributes using a constructor.
   - Provide setter and getter methods for both attributes.
   - Demonstrate updating and retrieving attribute values.
3. Create a class called `Rectangle` with attributes `width` and `height`.
   - Provide methods to calculate area and perimeter.
   - Display the calculated values.
4. Create a class called `Circle` with an attribute `radius`.
   - Provide getter and setter methods for radius.
   - Provide methods to calculate area and circumference.
   - Display the calculated values.
5. Create a class called `Book` with attributes `title`, `author`, and `ISBN`.
   - Maintain a list of books inside the class.
   - Provide methods to add and remove books from the collection.
6. Create a class called `Employee` with attributes `name`, `job_title`, and `salary`.
   - Provide a method to calculate yearly salary.
   - Provide a method to update salary with a raise.
7. Create two classes: `Bank` and `Account`.
   - The `Account` class should have attributes `account_number`, `holder_name`, and `balance`.
   - Include methods to deposit and withdraw money.
   - In the `Bank` class, maintain a list of accounts with methods to add and remove them.
8. Create a class called `TrafficLight` with attributes `color` and `duration`.
   - Provide a method to change the light color.
   - Provide methods to check whether the light is red or green.
9. Create a class called `Student` with attributes `name`, `ID`, and a list of `courses`.
   - Provide methods to add and remove courses.

- Display student details along with enrolled courses.

10. Create a class called `Library`.

- Maintain a collection of books (as list or dictionary).
- Provide methods to add, remove, and display books.

11. Create a class called `Flight` with attributes `flight_number`, `destination`, and `departure_time`.

- Provide methods to update flight status and delay departure time.

12. Create a class called `Inventory`.

- Maintain a dictionary of products with quantities.
- Provide methods to add, remove, and check for low inventory.

13. Create a class called `School`.

- Maintain lists of students and teachers.
- Provide methods to add and remove each.
- Provide a method to display all records.

14. Create a class called `MusicLibrary`.

- Maintain a list of songs.
- Provide methods to add, remove, and display songs.
- Provide a method to play a random song (use `random.choice`).

15. Create an abstract class called `Shape`.

- Define abstract methods `area()` and `perimeter()`.
- Create subclasses `Rectangle`, `Circle`, and `Triangle` that implement these methods.

16. Create a class called `Movie` with attributes `title`, `director`, and a list of `reviews`.

- Provide a method to add reviews.
- Provide a method to display all reviews.

17. Create a class called `Restaurant`.

- Maintain menu items and their prices in a dictionary.
- Provide methods to add/remove items.
- Provide a method to calculate the average price.

18. Create a class called `TravelBooking`.

- Provide methods to search for flights and hotels (use lists for sample data).
- Provide methods to book and cancel reservations.

19. Create a class called `BankAccount` with attributes `account_number`, `holder_name`, and `balance`.

- Provide methods to deposit, withdraw, and check balance.
- Create a subclass `SavingsAccount` with an attribute `interest_rate` and a method to apply interest.

20. Create a class called `Vehicle` with attributes `make`, `model`, and `year`.

- Create subclasses `Car` (with trunk size) and `Truck` (with payload capacity).
- Provide a method in each subclass to display details.

21. Create a class called `Customer` with attributes `name`, `email`, and `purchase_history`.

- Provide methods to add purchases and calculate total expenditure.
- Create a subclass `LoyalCustomer` with a `discount_rate` attribute and a method to apply discounts.

22. Create a class called `Course` with attributes `course_name`, `instructor`, and `credits`.

- Create a subclass `OnlineCourse` with attributes `platform` and `duration`.
- Provide methods to display details and check certificate eligibility.

23. Create a class called `Product` with attributes `product_id`, `name`, and `price`.

- Provide a method to apply discounts.
- Create a subclass `WashingMachine` with attribute `warranty_period` and a method to extend the warranty.

24. Create a class called `Building` with attributes `address`, `floors`, and `area`.

- Create subclasses `ResidentialBuilding` (with number of apartments) and `CommercialBuilding` (with office count).
- Provide methods to calculate total rent.

25. Create a class called `Event` with attributes `name`, `date`, and `location`.

- Create subclasses `Seminar` (with speaker count) and `Concert` (with performer list).
- Provide methods to display details and check scheduling conflicts.

26. Create a class called `Order` with attributes `order_id`, `customer_name`, and `date`.

- Create a subclass `OnlineOrder` with attributes `delivery_address` and `tracking_number`.
- Provide methods to calculate delivery time and update tracking status.

27. Create a class called `Reservation` with attributes `reservation_id`, `customer_name`, and `date`.

- Create subclasses `HotelReservation` (with room number) and `TrainReservation` (with seat number).

- Provide methods to check status and modify details.

28. Create a class called `Pet` with attributes `name`, `species`, and `age`.

- Create subclasses `Dog` (with favorite toy) and `Bird` (with wing span).
- Provide a method to calculate pet's age in human years.

29. Create a class called `GymMembership` with attributes `member_name`, `membership_type`, and `duration`.

- Create a subclass `PremiumMembership` with attributes `trainer_available` and `spa_access`.
- Provide methods to calculate membership fees and check for special offers.